SPRINT 03 · ANGULAR · LAB03

Forms

Template-Driven, Reactive Forms, FormArray, validadores y validadores personalizados

Ejercicios de entrenamiento. Hay tres niveles y cada uno es un proyecto distinto e independiente, así que no necesitas el anterior para empezar. Los enunciados no incluyen la solución: usa «Cómo saber que está bien» para comprobar tu trabajo.

Nivel	Proyecto	Practicas	Tiempo orientativo
Fácil	Reserva de restaurante	`ngForm`, `ngModel`, `ngModelGroup`, `ngModelChange` y mensajes de validación	1 h
Medio	Inscripción a carrera popular	Reactive Forms tipados, grupos anidados, `FormArray`, `valueChanges` y `patchValue`	2-2,5 h
Difícil	Solicitud de alquiler	Validadores personalizados, asíncronos y de grupo, validación condicional y formularios por pasos	4-5 h

> Trabaja con Angular 22, componentes standalone, `@if` / `@for` e `inject()`. Crea cada proyecto por separado con `ng new <nombre> --style=scss --ssr=false`. Si prefieres decoradores (`@Input`, `@Output`) o funciones (`input()`, `output()`), ambas opciones valen. Angular 22 no usa Zone.js: guarda en un `signal` el estado que cambie fuera de un evento de plantilla (temporizadores, HTTP o suscripciones al router), o la vista no se actualizará.

Fácil

1 h

`reserva-restaurante`

Reserva de restaurante

Practicas: `ngForm`, `ngModel`, `ngModelGroup`, `ngModelChange` y mensajes de validación

Crea el formulario de reservas de un restaurante con el enfoque Template-Driven.

Modelo

```
export interface Reserva {
  contacto: { nombre: string; telefono: string };
  fecha: string;
  comensales: number | null;
  zona: 'interior' | 'terraza';
  notas: string;
}
```

Requisitos

1. Agrupa `nombre` y `telefono` con `ngModelGroup="contacto"`.
2. `nombre`: obligatorio, mínimo 2 caracteres. `telefono`: obligatorio, con el patrón `^[0-9]{9}$`.
3. `fecha` y `comensales` son obligatorios. `comensales` se elige en un `<select>` del 1 al 12.
4. `zona` usa botones de radio y empieza en «interior». `notas` es opcional.
5. Exporta cada control a una referencia (`#nombre="ngModel"`). Muestra un mensaje distinto para cada error, solo si el campo es inválido y ya se ha tocado o modificado.
6. Con `(ngModelChange)`, si hay más de 8 comensales, muestra: «Para grupos, confirmaremos por teléfono».
7. Desactiva «Reservar» si el formulario es inválido. Al enviar, muestra el objeto en consola y resetea con `form.resetForm()`, volviendo a los valores iniciales.

Cómo saber que está bien

- Nada más cargar la página no se ve ningún error.
- La consola muestra `contacto` como un objeto anidado.
- Tras enviar, el formulario queda limpio, sin errores y con «interior» seleccionado.

Preguntas

- ¿Qué diferencia hay entre `touched`, `dirty` y `invalid`?
- ¿Por qué en `<select>` numéricos conviene `[ngValue]` en lugar de `value`?

Medio

2-2,5 h

`carrera-popular`

Inscripción a carrera popular

Practicas: Reactive Forms tipados, grupos anidados, `FormArray`, `valueChanges` y `patchValue`

En una carrera popular, una misma persona puede inscribir a sus acompañantes. El precio se recalcula conforme se rellena el formulario.

Campos

Campo	Inicial	Validación
titular.nombre	''	Obligatorio, mínimo 3 caracteres
titular.email	''	Obligatorio, formato de email
titular.edad	null	Obligatorio, entre 16 y 90
distancia	'5k'	Obligatorio: 5k, 10k o 21k
camiseta	false	Opcional
acompanantes	[]	Máximo 4; nombre obligatorio y edad entre 6 y 90

Precios

Concepto	Precio
Dorsal 5k / 10k / 21k	10 € / 15 € / 25 €, por persona
Acompañante menor de 16 años	50 % de descuento en su dorsal
Camiseta	8 €, una sola vez

Requisitos

1. Crea el formulario con `FormBuilder` (usa `nonNullable` donde convenga) y agrupa los datos del titular.
2. `acompanantes` es un `FormArray` de `FormGroup`. Añade botones para añadir y quitar. Desactiva «Añadir» al llegar a 4.
3. Calcula el precio a partir de `valueChanges` y actualízalo con cada cambio. Comprueba con `startWith` que el precio inicial es correcto.
4. Cada campo muestra un mensaje específico por error. Crea un componente `ErrorCampo` reutilizable para no repetir código.
5. Al pulsar «Inscribirme» con errores, usa `markAllAsTouched()` para mostrarlos todos a la vez.
6. Añade «Cargar mis datos», que rellena solo el titular usando `patchValue`.
7. Al enviar, muestra en consola `getRawValue()` y el precio final.

Cómo saber que está bien

- Titular en 10k con camiseta y un acompañante de 12 años: 15 + 8 + 7,50 = **30,50 €**.
- Quitar un acompañante actualiza el precio de inmediato.
- Un acompañante vacío hace que todo el formulario sea inválido.

Preguntas

- ¿Qué hace `setValue` si le pasas solo una parte del grupo? ¿Y `patchValue`?
- ¿Qué diferencia hay entre `value` y `getRawValue()` cuando un control está desactivado?

Difícil

4-5 h

`solicitud-alquiler`

Solicitud de alquiler

Practicas: Validadores personalizados, asíncronos y de grupo, validación condicional y formularios por pasos

Una inmobiliaria necesita un asistente de tres pasos para solicitar un alquiler. Cada paso tiene su `FormGroup` dentro de un formulario común. Usa solo datos inventados.

Pasos

Paso	Campos
1. Inquilino	nombre, nif, email
2. Contrato	fechaInicio, fechaFin, rentaMensual, ingresosMensuales, avalista
3. Referencias	referencias: lista de 1 a 3 con nombre y telefono

Requisitos

1. `nifValido()` es un `ValidatorFn` propio. Comprueba 8 dígitos y la letra: es la posición `numero % 23` de `TRWAGMYFPDXBNJZSQVHLCKE`. `12345678Z` es válido; `12345678A` no.
2. `emailDisponible()` es un `AsyncValidatorFn` que espera 800 ms con `timer` y rechaza `ocupado@ejemplo.com`. Mientras `pending` sea verdadero, muestra «Comprobando…». Usa `updateOn: 'blur'`.
3. `rangoFechas('fechaInicio', 'fechaFin')` es un validador de grupo. Devuelve `{ rangoInvalido: true }` si la fecha de fin no es posterior.
4. **Validación condicional:** si los ingresos son menores que tres veces la renta, `avalista` pasa a ser obligatorio. Si no, se limpia su valor y deja de validarse. Usa `setValidators` o `clearValidators` y `updateValueAndValidity`.
5. «Siguiente» solo avanza si el paso actual es válido; si no, marca sus campos como tocados. «Atrás» conserva los datos.
6. El último paso muestra un resumen de solo lectura antes de enviar.
7. Guarda un borrador en `localStorage` al cambiar un paso y restáuralo al recargar. Bórralo al enviar.

Cómo saber que está bien

- Con `ocupado@ejemplo.com`, el botón «Siguiente» no avanza y aparece el error tras la comprobación.
- Renta de 1000 € e ingresos de 2500 € exige avalista; con ingresos de 3000 €, deja de exigirlo y el campo queda vacío.
- Recargar en el paso 2 recupera los datos del borrador.

Preguntas

- ¿Por qué el error de fechas va en el grupo y no en `fechaFin`?
- ¿Por qué no se ejecuta un validador asíncrono mientras fallen los síncronos?
- ¿Qué problema de privacidad tiene guardar un NIF en `localStorage`? ¿Qué campos excluirías del borrador en una aplicación real?

Reto opcional

> En clase se presentaron los Signal Forms. Rehaz el paso 1 con `@angular/forms/signals`, consultando la documentación oficial, y compara cuánto código necesita cada enfoque.